

CO5_ASSESSMENTS 2
PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

CO5 – ASSESSMENT 2

PSEUDOCODE WRITING TASK

DETAILED OPENCV-BASED SOLUTIONS

Student Name: K Nitheesh Kumar

Register Number: 192325090

Assignment Overview

The supplied task contains two pseudocode problems. Question 1 requires a complete real-time face detection and recognition pipeline that continuously captures frames, detects multiple faces, recognizes registered persons using a trained dataset, displays bounding boxes and identity labels, and minimizes processing delay. Question 2 requires vehicle detection, multi-object tracking and counting across a predefined region and virtual line while avoiding double counting and maintaining near real-time performance. 穀filecite學turn15file0學L2-L14係

Answering Strategy

The answers below are written as detailed implementation-oriented pseudocode. Each solution is decomposed into initialization, video acquisition, preprocessing, detection, tracking or feature extraction, decision logic, visualization, error handling and efficient termination. The pseudocode is language-independent but uses OpenCV-style function names so that it can be translated into Python, C++ or another OpenCV-supported language.

Student Details

Field	Value
Student Name	K Nitheesh Kumar
Register Number	192325090
Assessment	CO5 Assessment 2 – Pseudocode Writing Task
Technology	OpenCV / Computer Vision
Signature	K Nitheesh Kumar

Question Coverage

Question	Topic	Required Components
1	Real-Time Face Detection and Recognition	Acquisition, preprocessing, detection, feature extraction, recognition, visualization, efficient loop.
2	Vehicle Detection, Tracking and Counting	Acquisition, preprocessing, detection, tracking, counting, visualization, efficient termination.

QUESTION 1 – REAL-TIME FACE DETECTION AND RECOGNITION

The first problem statement requires a security system to process a live camera stream continuously, detect multiple faces, recognize registered persons using a trained dataset and display bounding boxes and identity labels. Its constraints are real-time operation, minimum processing delay and simultaneous handling of multiple faces. 穀filecite學turn15file0學L4-L8傢

1.1 Complete Pipeline

```
START
|
+--> Load configuration
|
+--> Load trained face recognition dataset/model
|
+--> Load face detector
|
+--> Open camera/video stream
|
+--> LOOP while stream is available
|   |
|   +--> Capture frame
|   +--> Validate frame
|   +--> Resize / preprocess
|   +--> Detect all faces
|   +--> For each detected face
|   |   |
|   |   +--> Crop face
|   |   +--> Align / normalize
|   |   +--> Extract features
|   |   +--> Compare with registered dataset
|   |   +--> Apply confidence threshold
|   |   +--> Assign identity / Unknown
|   |   +--> Draw box and label
|   |
|   +--> Display processed frame
|   +--> Check exit key
|
+--> Release camera
+--> Close windows
END
```

1.2 Initialization Pseudocode

```
INITIALIZE system
    SET camera_source = default_camera
    SET frame_width = target_width
    SET frame_height = target_height
    SET recognition_threshold = configured_value
    SET detection_interval = configured_value
    SET minimum_face_size = configured_value

    LOAD face_detector
    IF detector cannot be loaded:
        PRINT "Face detector initialization failed"
        TERMINATE

    LOAD recognition_model
    IF recognition_model cannot be loaded:
        PRINT "Recognition model initialization failed"
        TERMINATE

    LOAD registered_embeddings / trained_dataset
    IF dataset is empty:
        PRINT "No registered identities available"
        TERMINATE

    OPEN camera_source
```

```
IF camera is not opened:
    PRINT "Camera unavailable"
    TERMINATE

INITIALIZE frame_counter = 0
INITIALIZE previous_faces = empty
INITIALIZE tracker_state = empty
END INITIALIZE
```

1.3 Why Initialization Matters

Loading the detector, recognition model and registered dataset once before entering the frame loop avoids repeated initialization overhead. Configuration values such as thresholds, frame size and detection interval should also be established once so that the loop performs only the work necessary for each frame.

QUESTION 1 – IMAGE ACQUISITION AND PREPROCESSING

1.4 Image Acquisition

```
WHILE camera is open:

    success, frame = READ_NEXT_FRAME()

    IF success == FALSE:
        PRINT "Frame acquisition failed"
        BREAK

    frame_counter = frame_counter + 1

    STORE original_frame_size

    IF frame_counter is suitable for detection:
        SEND frame to preprocessing
    ELSE:
        USE previous detections / tracker state
END WHILE
```

1.5 Preprocessing Pseudocode

```
FUNCTION PREPROCESS_FRAME(frame):

    IF frame is NULL:
        RETURN failure

    resized_frame = RESIZE(
        frame,
        width = configured_width,
        height = configured_height
    )

    OPTIONAL:
        denoised_frame = APPLY_GAUSSIAN_OR_MEDIAN_FILTER(
            resized_frame
        )

    gray_frame = CONVERT_TO_GRAYSCALE(resized_frame)

    OPTIONAL:
        normalized_frame = NORMALIZE_ILLUMINATION(gray_frame)

    RETURN preprocessed_frame
END FUNCTION
```

1.6 Preprocessing Explanation

Operation	Purpose	Caution
Resize	Reduces computation and latency.	Do not make faces too small.
Grayscale	Simplifies intensity-based detection.	Use color if detector requires it.
Denoising	Reduces camera noise.	Excessive filtering can remove facial detail.
Illumination normalization	Improves robustness under lighting variation.	Avoid creating artificial artifacts.
Alignment	Normalizes face orientation before recognition.	Requires reliable landmarks if used.

1.7 Quality Checks

- Reject frames that were not captured successfully.
- Check that the image dimensions are valid.
- Ignore face regions below the configured minimum size.
- Skip severely blurred regions when quality estimation is available.
- Do not allow invalid crops to reach the feature-extraction stage.

QUESTION 1 – FACE DETECTION AND MULTIPLE-FACE HANDLING

1.8 Face Detection Pseudocode

```
FUNCTION DETECT_FACES(preprocessed_frame):

    detections = face_detector.DETECT(preprocessed_frame)

    valid_faces = EMPTY_LIST

    FOR each detection IN detections:

        x1, y1, x2, y2, confidence = detection

        IF confidence < detection_threshold:
            CONTINUE

        face_width = x2 - x1
        face_height = y2 - y1

        IF face_width < minimum_face_size:
            CONTINUE

        IF face_height < minimum_face_size:
            CONTINUE

        CLIP coordinates to frame boundaries

        ADD valid face bounding box to valid_faces

    RETURN valid_faces
END FUNCTION
```

1.9 Multiple Faces

The detector must return a collection of face regions rather than a single region. The system processes each region independently while preserving its bounding-box coordinates. This allows several registered or unknown people to appear in the same frame.

1.10 Non-Maximum Suppression / Duplicate Detections

If the selected detector can return overlapping detections, duplicate boxes should be removed using a confidence-based suppression step. The goal is to keep one reliable bounding box per physical face.

```
detections = DETECT_FACES(frame)

IF detector_returns_overlapping_boxes:
    detections = NON_MAXIMUM_SUPPRESSION(
        detections,
        overlap_threshold
    )

FOR face_box IN detections:
    PROCESS_FACE(face_box)
```

1.11 Face Crop and Alignment

```
FUNCTION PREPARE_FACE(frame, bounding_box):

    crop = CROP(frame, bounding_box)

    IF crop is too small:
```

```

        RETURN invalid

OPTIONAL landmarks = DETECT_FACE_LANDMARKS(crop)

IF landmarks are available:
    aligned = ALIGN_FACE_USING_LANDMARKS(
        crop,
        landmarks,
        standard_face_template
    )
ELSE:
    aligned = RESIZE(crop, model_input_size)

normalized = NORMALIZE_PIXEL_VALUES(aligned)

RETURN normalized
END FUNCTION

```

1.12 Processing Order

Detection must happen before recognition because the recognition model needs a face region. Alignment and normalization should occur between detection and feature extraction so that registered and live face representations use comparable input conditions.

QUESTION 1 – FEATURE EXTRACTION AND RECOGNITION

1.13 Feature Extraction

```
FUNCTION EXTRACT_FEATURES(prepared_face):  
  
    IF prepared_face is invalid:  
        RETURN failure  
  
    feature_vector = recognition_model.ENCODE(  
        prepared_face  
    )  
  
    feature_vector = NORMALIZE_VECTOR(feature_vector)  
  
    RETURN feature_vector  
END FUNCTION
```

1.14 Recognition Pseudocode

```
FUNCTION RECOGNIZE_PERSON(feature_vector, registered_data):  
  
    best_identity = "Unknown"  
    best_score = INFINITY  
  
    FOR each identity IN registered_data:  
  
        FOR each stored_vector IN identity.embeddings:  
  
            distance = CALCULATE_DISTANCE(  
                feature_vector,  
                stored_vector  
            )  
  
            IF distance < best_score:  
                best_score = distance  
                best_identity = identity.name  
  
    IF best_score <= recognition_threshold:  
        RETURN best_identity, best_score  
    ELSE:  
        RETURN "Unknown", best_score  
END FUNCTION
```

1.15 Recognition Decision

The recognition stage compares the live feature vector with stored representations of registered individuals. The nearest candidate is not automatically accepted. A threshold is required because an unknown person can be closer to a registered identity than other candidates. The threshold should be selected using validation data.

1.16 Temporal Confirmation

```
FUNCTION STABILIZE_IDENTITY(track_id, candidate_identity, score):  
  
    UPDATE identity_history[track_id] with candidate_identity  
  
    IF same identity appears consistently  
        for required_consecutive_frames:  
  
        stable_identity = candidate_identity  
  
    ELSE:
```

```
        stable_identity = previous_stable_identity  
        OR "Unknown"  
  
    RETURN stable_identity  
END FUNCTION
```

1.17 Why Temporal Confirmation Helps

- Reduces single-frame false recognitions.
- Allows brief detector noise without immediately changing identity.
- Provides more stable labels in the displayed video.
- Supports tracking-based recognition so expensive feature extraction need not run every frame.

QUESTION 1 – VISUALIZATION AND EFFICIENT LOOP CONTROL

1.18 Visualization Pseudocode

```
FUNCTION DRAW_RESULT(frame, face_box, identity, score):

    x1, y1, x2, y2 = face_box

    DRAW_RECTANGLE(
        frame,
        top_left = (x1, y1),
        bottom_right = (x2, y2)
    )

    label = identity + FORMAT_SCORE(score)

    DRAW_TEXT(
        frame,
        label,
        position = (x1, y1 - label_offset)
    )

    RETURN frame
END FUNCTION
```

1.19 Complete Face Processing Loop

```
OPEN camera

WHILE TRUE:

    success, frame = camera.READ()

    IF success == FALSE:
        BREAK

    frame_counter += 1

    IF frame_counter MOD detection_interval == 0:

        processed = PREPROCESS_FRAME(frame)

        face_boxes = DETECT_FACES(processed)

        UPDATE_TRACKS(face_boxes)

        FOR each track IN active_tracks:

            face = PREPARE_FACE(frame, track.bounding_box)

            IF face is invalid:
                CONTINUE

            vector = EXTRACT_FEATURES(face)

            identity, score = RECOGNIZE_PERSON(
                vector,
                registered_data
            )
```

```

        identity = STABILIZE_IDENTITY(
            track.id,
            identity,
            score
        )

        DRAW_RESULT(
            frame,
            track.bounding_box,
            identity,
            score
        )

    ELSE:

        UPDATE_TRACKER_WITH_CURRENT_FRAME()

        DRAW_EXISTING_TRACK_RESULTS(frame)

    DISPLAY(frame)

    key = READ_KEY_WITH_SHORT_DELAY()

    IF key == ESCAPE OR key == 'q':
        BREAK

RELEASE camera
DESTROY all display windows
END

```

1.20 Efficient Loop Control

- Use frame skipping or configurable detection intervals.
- Track faces between expensive recognition operations.
- Resize frames before detection.
- Process only detected face crops during recognition.
- Keep model loading outside the loop.
- Use a short display wait so the UI remains responsive.
- Terminate immediately when the user presses the configured exit key.
- Release the camera and windows even after an error.

These controls directly address the task's real-time and minimum-delay constraints. 穀filecite學turn15file0學L5-L8像

QUESTION 1 – COMPLETE INTEGRATED PSEUDOCODE

1.21 End-to-End Pseudocode

ALGORITHM REAL_TIME_FACE_RECOGNITION

INPUT:

camera_source
registered_face_dataset
detection_model
recognition_model
recognition_threshold
detection_threshold
target_resolution
detection_interval

OUTPUT:

Live video with face boxes and identity labels

BEGIN

LOAD detection_model
LOAD recognition_model
LOAD registered_face_dataset

IF any required resource is unavailable:
 REPORT initialization error
 RETURN

camera = OPEN_VIDEO_CAPTURE(camera_source)

IF camera is unavailable:
 REPORT camera error
 RETURN

INITIALIZE tracker
INITIALIZE identity_history
INITIALIZE frame_counter = 0

WHILE TRUE:

 ok, frame = camera.READ()

 IF NOT ok:
 BREAK

 frame_counter = frame_counter + 1

 frame_for_detection = RESIZE(frame, target_resolution)

 IF frame_counter MOD detection_interval == 0:

 processed = PREPROCESS_FRAME(
 frame_for_detection
)

 faces = DETECT_FACES(processed)

 tracks = UPDATE_TRACKER(faces)

```

FOR each track IN tracks:

    face_crop = PREPARE_FACE(
        frame_for_detection,
        track.bounding_box
    )

    IF face_crop is invalid:
        CONTINUE

    features = EXTRACT_FEATURES(face_crop)

    identity, distance = RECOGNIZE_PERSON(
        features,
        registered_face_dataset
    )

    stable_identity = STABILIZE_IDENTITY(
        track.id,
        identity,
        distance
    )

    DRAW_RESULT(
        frame_for_detection,
        track.bounding_box,
        stable_identity,
        distance
    )

ELSE:

    tracks = UPDATE_TRACKER_WITHOUT_FULL_DETECTION()

    DRAW_TRACKS_AND_LAST_IDENTITIES(
        frame_for_detection,
        tracks
    )

    DISPLAY frame_for_detection

    key = WAIT_FOR_KEY()

    IF key == ESCAPE OR key == 'q':
        BREAK

RELEASE camera
CLOSE all windows

END

```

1.22 Expected Output

The output window shows the live camera stream. Each detected face has a bounding box. A recognized registered person is displayed with the matched identity label, while a low-confidence or unmatched face is displayed as Unknown. The system continues processing until the stream ends or the user presses the configured termination key.

1.23 Complexity Considerations

If F faces are detected and R registered feature vectors are compared for each face, direct matching requires approximately $O(F \times R)$ distance computations per recognition cycle. Reducing F through region-of-interest constraints, reducing recognition frequency through tracking, or using an indexed/optimized representation of registered embeddings can improve performance.

1.24 Error Handling

- Camera failure → stop safely and release resources.
- Invalid frame → skip frame or terminate after repeated failures.
- Detector failure → report error and stop or fall back to configured detector.
- Recognition model failure → do not produce false identities.
- Unknown face → display Unknown instead of forcing a registered identity.
- Low-quality face → skip recognition for that region.

QUESTION 2 – VEHICLE DETECTION, TRACKING AND COUNTING

The second problem requires a traffic monitoring system to detect vehicles entering a predefined region of interest, track multiple vehicles across consecutive frames, count vehicles crossing a virtual line, and display bounding boxes, tracking IDs and vehicle count. The constraints are avoiding double counting, handling multiple vehicles simultaneously and maintaining near real-time performance. 穀filecite學turn15file0學L9-L14傢

2.1 Complete Pipeline

```
Live Traffic Camera
↓
Frame Acquisition
↓
Resize / Preprocessing
↓
Region of Interest
↓
Vehicle Detection
↓
Bounding Boxes
↓
Multi-Object Tracker
↓
Centroid / Track History
↓
Virtual Line Crossing Test
↓
One-Time Count Update
↓
Visualization
↓
Efficient Loop / Exit
```

2.2 Initialization Pseudocode

```
INITIALIZE traffic_system

SET camera_source
SET target_resolution
SET detection_interval
SET detection_confidence
SET minimum_vehicle_size
SET tracking_distance_threshold
SET max_missing_frames

DEFINE region_of_interest
DEFINE virtual_counting_line

LOAD vehicle_detector

IF detector unavailable:
    REPORT error
    TERMINATE

OPEN traffic_camera

IF camera unavailable:
    REPORT error
    TERMINATE

INITIALIZE tracker
INITIALIZE next_track_id = 1
INITIALIZE active_tracks = empty
INITIALIZE total_vehicle_count = 0
```


END INITIALIZE

2.3 Data Structure for Each Track

Field	Purpose
track_id	Unique identifier for one vehicle.
bbox	Current vehicle bounding box.
centroid	Current object center.
previous_centroid	Previous center for movement comparison.
last_seen	Frame/time of latest detection.
missing_frames	Number of frames without matching detection.
counted	Prevents the same vehicle from being counted twice.
class	Optional vehicle category.
confidence	Latest detection confidence.

QUESTION 2 – VIDEO ACQUISITION, PREPROCESSING AND DETECTION

2.4 Frame Acquisition

```
WHILE camera is open:

    ok, frame = camera.READ()

    IF ok == FALSE:
        PRINT "End of stream or camera error"
        BREAK

    frame_counter += 1

    IF frame is empty:
        CONTINUE

    PROCESS frame
END WHILE
```

2.5 Preprocessing

```
FUNCTION PREPROCESS_TRAFFIC_FRAME(frame):

    resized = RESIZE(
        frame,
        target_width,
        target_height
    )

    blurred = GAUSSIAN_BLUR(
        resized,
        kernel_size
    )

    roi_frame = APPLY_REGION_MASK(
        blurred,
        region_of_interest
    )

    RETURN roi_frame
END FUNCTION
```

2.6 Vehicle Detection

```
FUNCTION DETECT_VEHICLES(frame):

    raw_detections = vehicle_detector.DETECT(frame)

    valid_detections = EMPTY_LIST

    FOR detection IN raw_detections:

        class_id = detection.class_id
        confidence = detection.confidence
        bbox = detection.bounding_box

        IF class_id NOT IN allowed_vehicle_classes:
            CONTINUE
```

```

    IF confidence < detection_confidence:
        CONTINUE

    IF bbox.width < minimum_vehicle_width:
        CONTINUE

    IF bbox.height < minimum_vehicle_height:
        CONTINUE

    IF NOT INTERSECTS_ROI(bbox, region_of_interest):
        CONTINUE

    centroid = CALCULATE_CENTROID(bbox)

    ADD (bbox, centroid, class_id, confidence)
    TO valid_detections

valid_detections =
    REMOVE_OVERLAPPING_DUPLICATES(valid_detections)

RETURN valid_detections
END FUNCTION

```

2.7 Why ROI is Important

Restricting detection to the roadway reduces computation and prevents objects outside the monitored lanes from becoming candidates. The ROI also makes counting rules more reliable because the system only tracks vehicles relevant to the traffic-monitoring area.

QUESTION 2 – MULTI-OBJECT TRACKING AND COUNTING LOGIC

2.8 Track Association

```
FUNCTION UPDATE_TRACKS(detections):

    MARK all existing tracks as unmatched

    FOR each detection IN detections:

        best_track = NONE
        best_distance = INFINITY

        FOR each active_track IN active_tracks:

            distance = EUCLIDEAN_DISTANCE(
                detection.centroid,
                active_track.centroid
            )

            IF distance < best_distance
                AND distance < tracking_distance_threshold:

                best_distance = distance
                best_track = active_track

        IF best_track exists:

            best_track.previous_centroid =
                best_track.centroid

            best_track.centroid =
                detection.centroid

            best_track.bbox =
                detection.bbox

            best_track.confidence =
                detection.confidence

            best_track.missing_frames = 0
            MARK best_track as matched

        ELSE:

            CREATE new_track
            new_track.track_id = next_track_id
            next_track_id += 1
            new_track.previous_centroid = detection.centroid
            new_track.centroid = detection.centroid
            new_track.bbox = detection.bbox
            new_track.counted = FALSE
            new_track.missing_frames = 0

            ADD new_track to active_tracks

    FOR each track NOT matched:
        track.missing_frames += 1
```

```

        IF track.missing_frames > max_missing_frames:
            REMOVE track from active_tracks

    RETURN active_tracks
END FUNCTION

```

2.9 Virtual Line Crossing

```

FUNCTION CHECK_LINE_CROSSING(track):

    previous_point = track.previous_centroid
    current_point = track.centroid

    IF previous_point is undefined:
        RETURN FALSE

    previous_side =
        DETERMINE_SIDE_OF_LINE(
            previous_point,
            virtual_counting_line
        )

    current_side =
        DETERMINE_SIDE_OF_LINE(
            current_point,
            virtual_counting_line
        )

    IF previous_side != current_side:

        IF track.counted == FALSE:

            track.counted = TRUE

            RETURN TRUE

    RETURN FALSE
END FUNCTION

```

2.10 Counting Update

```

FOR each track IN active_tracks:

    IF CHECK_LINE_CROSSING(track) == TRUE:

        total_vehicle_count =
            total_vehicle_count + 1

        RECORD count_event(
            track_id = track.track_id,
            timestamp = CURRENT_TIME()
        )

```

2.11 Why Double Counting is Avoided

Every track contains a counted flag. Once the vehicle crosses the virtual line and the count is incremented, the flag becomes true. Future frames for the same track cannot increment the total again. If direction-specific counting is required, the system can additionally store the previous and current side of the line and count only a specified direction.

QUESTION 2 – VISUALIZATION, OPTIMIZATION AND FINAL PSEUDOCODE

2.12 Visualization Pseudocode

```
FUNCTION DRAW_TRAFFIC_OUTPUT(frame, tracks, count):

    DRAW_POLYGON(
        frame,
        region_of_interest
    )

    DRAW_LINE(
        frame,
        virtual_counting_line
    )

    FOR each track IN tracks:

        DRAW_RECTANGLE(
            frame,
            track.bbox
        )

        label =
            "ID: " + track.track_id

        IF track.class is available:
            label += " " + track.class

        DRAW_TEXT(
            frame,
            label,
            track.bbox.top_left
        )

        DRAW_CENTROID(
            frame,
            track.centroid
        )

    DRAW_TEXT(
        frame,
        "Vehicle Count: " + count,
        dashboard_position
    )

    RETURN frame
END FUNCTION
```

2.13 Efficient Loop Termination and Optimization

- Use a reduced processing resolution appropriate to the camera.
- Run expensive vehicle detection at a controlled interval.
- Use the tracker between detector updates.
- Restrict processing to the ROI.
- Filter detections before creating tracks.
- Remove stale tracks after a configurable missing-frame limit.
- Use efficient data structures for active tracks.

- Display the frame with a short wait to keep the interface responsive.
- Terminate on ESC/q or when the camera stream ends.
- Release the camera and destroy windows in all termination paths.

2.14 Complete Integrated Pseudocode

ALGORITHM VEHICLE_DETECTION_TRACKING_COUNTING

INPUT:

```

traffic_camera
vehicle_detector
region_of_interest
virtual_counting_line
detection_interval
confidence_threshold
tracking_distance_threshold
max_missing_frames

```

OUTPUT:

Live video with boxes, track IDs and vehicle count

BEGIN

OPEN traffic_camera

```

IF camera unavailable:
    REPORT error
    RETURN

```

LOAD vehicle_detector

```

INITIALIZE tracker
INITIALIZE active_tracks = empty
INITIALIZE next_track_id = 1
INITIALIZE vehicle_count = 0
INITIALIZE frame_counter = 0

```

WHILE TRUE:

```

    ok, frame = READ_FRAME()

```

```

    IF ok == FALSE:
        BREAK

```

```

    frame_counter += 1

```

```

    frame = RESIZE(frame)

```

```

    IF frame_counter MOD detection_interval == 0:

```

```

        processed = PREPROCESS_TRAFFIC_FRAME(frame)

```

```

        detections = DETECT_VEHICLES(processed)

```

```

        active_tracks =
            UPDATE_TRACKS(detections)

```

```

        FOR each track IN active_tracks:

```

```

        IF CHECK_LINE_CROSSING(track):
            vehicle_count += 1
            LOG track.track_id and current time

    ELSE:

        active_tracks =
            UPDATE_TRACKER_FROM_FRAME()

        output =
            DRAW_TRAFFIC_OUTPUT(
                frame,
                active_tracks,
                vehicle_count
            )

        DISPLAY output

        key = WAIT_FOR_KEY()

        IF key == ESCAPE OR key == 'q':
            BREAK

    RELEASE camera
    DESTROY all windows

END

```

2.15 Expected Output

The output displays the traffic video, the predefined ROI, the virtual counting line, bounding boxes around detected vehicles, a unique tracking ID for each active vehicle and the total count. Each vehicle contributes to the count only once when its tracked position crosses the line.

3. COMPARATIVE ANALYSIS OF THE TWO PSEUDOCODE SOLUTIONS

Aspect	Face Recognition	Vehicle Tracking & Counting
Input	Live camera stream	Live traffic video
Detection target	Multiple faces	Multiple vehicles
Feature stage	Face alignment + feature embedding	Bounding box + centroid/motion
Identity	Registered-person matching	Track ID
Temporal logic	Identity stabilization	Track persistence
Final decision	Name / Unknown	Count / crossing event
Main error risk	False recognition	Double counting / ID switches
Optimization	Skip recognition frames + tracking	Skip detection frames + tracking
Output	Boxes + identity labels	Boxes + IDs + vehicle count
Termination	ESC/q or stream end	ESC/q or stream end

3.1 Common Design Pattern

Both solutions follow the same high-level computer-vision pattern: acquire a frame, preprocess it, detect relevant objects, maintain state across frames, apply an application-specific decision rule, visualize the result and terminate efficiently. The difference is the state stored for each object. Face recognition stores identity evidence, whereas vehicle counting stores track history and a counted state.

3.2 Important State Variables

System	Essential State
Face recognition	Track ID, bounding box, feature/identity history, confidence, last seen.
Vehicle counting	Track ID, current/previous centroid, bounding box, missing frames, counted flag.

3.3 Test Cases

Test	Expected Result
One known face	Correct identity shown after threshold confirmation.
Unknown face	Unknown label, no forced registered identity.
Several known faces	Each face receives an independent label.
Temporary face disappearance	Track can survive short disappearance.
One vehicle crossing line	Count increases exactly once.
Several vehicles crossing	Each distinct track counted once.
Vehicle oscillates near line	Count flag prevents repeated counting.
Camera stream ends	Loop terminates and resources release.

3.4 Final Quality Criteria

- Correct detection under representative conditions.

- Stable outputs across consecutive frames.
- No repeated counting or attendance-like duplicate events.
- Low enough latency for the intended application.
- Graceful handling of camera/model errors.
- Clean resource release at termination.

4. CONCLUSION

The detailed pseudocode satisfies the two tasks in the supplied CO5 Assessment 2. The face-recognition solution covers image acquisition, preprocessing, multi-face detection, feature extraction, recognition, thresholding, identity stabilization, visualization and efficient loop control. The vehicle solution covers acquisition, ROI preprocessing, vehicle detection, multi-object tracking, virtual line crossing, one-time counting, visualization and efficient termination. The source explicitly identifies these components and constraints, and the pseudocode maps each required item to a concrete processing step.

Student Details

Name: K Nitheesh Kumar

Register No.: 192325090

Signature: K Nitheesh Kumar